JAVA EDITION

DSA Interview Questions

Most Frequently Asked Data Structures & Algorithms Questions with Answers, Code, Tricks & Complexity Analysis

59	9	Java
QUESTIONS	TOPICS	LANGUAGE

Compiled July 13, 2026

Table of Contents

ARRAYS

STRINGS

LINKED LIST

STACK & QUEUE

TREES

28. Inorder / Preorder / Postorder Traversal (Recursive) — Easy
29. Level Order Traversal (BFS) — Easy
30. Find the Height / Maximum Depth of a Binary Tree — Easy
31. Check if a Binary Tree is Balanced — Medium
32. Lowest Common Ancestor (LCA) in a Binary Tree — Medium
33. Validate Binary Search Tree (BST) — Medium
34. Diameter of a Binary Tree — Medium
35. Serialize and Deserialize a Binary Tree — Hard

GRAPHS

36. Graph Representation (Adjacency List) — Easy
37. BFS Traversal of a Graph — Easy
38. DFS Traversal of a Graph — Easy
39. Detect Cycle in a Directed Graph — Medium
40. Topological Sort (Kahn's Algorithm — BFS based) — Medium
41. Number of Islands (Connected Components in a Grid) — Medium
42. Dijkstra's Algorithm (Shortest Path) — Medium

SORTING

43. Bubble Sort — Easy
44. Quick Sort — Medium
45. Merge Sort — Medium
46. Binary Search — Easy
47. Search in a Rotated Sorted Array — Medium
48. Find First and Last Position of Element (Binary Search Variant) — Medium
49. Sort an Array of 0s, 1s, and 2s (Dutch National Flag) — Medium

DYNAMIC PROGRAMMING

50. Fibonacci Number (Memoization & Tabulation) — Easy
51. 0/1 Knapsack Problem — Medium
52. Longest Common Subsequence (LCS) — Medium
53. Coin Change (Minimum Coins) — Medium
54. Longest Increasing Subsequence (LIS) — Medium
55. Climbing Stairs (Distinct Ways) — Easy

RECURSION & BACKTRACKING

56. Generate All Subsets (Power Set) — Medium

Arrays

Q1 Find the Maximum and Minimum Element in an Array

Easy

TRICK / KEY INSIGHT

Just track two variables in a single pass — never sort just to find min/max, that wastes O(n log n).

JAVA CODE

```java
public class MaxMin {
    public static void findMaxMin(int[] arr) {
        int max = arr[0], min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
            if (arr[i] < min) min = arr[i];
        }
        System.out.println("Max: " + max + ", Min: " + min);
    }

    public static void main(String[] args) {
        int[] arr = {3, 5, 1, 9, 2};
        findMaxMin(arr);
    }
}
```

EXPLANATION

Traverse the array once, updating max and min whenever a bigger or smaller value is found. Avoid sorting (O(n log n)) since a single linear pass (O(n)) is sufficient.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q2 Reverse an Array (In-Place)

Easy

TRICK / KEY INSIGHT

Two-pointer swap: left pointer starts at 0, right pointer at n-1, swap and move inward until they cross.

JAVA CODE

```java
public class ReverseArray {
    public static void reverse(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        reverse(arr);
        for (int x : arr) System.out.print(x + " ");
    }
}
```

EXPLANATION

Classic two-pointer technique. Swap elements from both ends moving toward the center. This is an in-place reversal, no extra array needed.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q3 Kadane's Algorithm — Maximum Subarray Sum

Medium

TRICK / KEY INSIGHT

The trick: at each index decide 'extend previous subarray OR start fresh here' — maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]).

JAVA CODE

```java
public class KadaneAlgorithm {
    public static int maxSubArraySum(int[] arr) {
        int maxSoFar = arr[0];
        int maxEndingHere = arr[0];

        for (int i = 1; i < arr.length; i++) {
            maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]);
            maxSoFar = Math.max(maxSoFar, maxEndingHere);
        }
        return maxSoFar;
    }

    public static void main(String[] args) {
        int[] arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        System.out.println("Max Subarray Sum: " + maxSubArraySum(arr));
    }
}
```

EXPLANATION

Dynamic programming approach. maxEndingHere tracks the best subarray sum ending at the current index; maxSoFar tracks the global best seen so far. This is one of the most frequently asked DP-on-array questions.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q4 Move All Zeros to the End

Easy

TRICK / KEY INSIGHT

Maintain a 'write pointer' for the next non-zero slot; single pass, swap non-zero elements forward.

JAVA CODE

```java
public class MoveZeros {
    public static void moveZeroes(int[] arr) {
        int insertPos = 0;
        for (int num : arr) {
            if (num != 0) {
                arr[insertPos++] = num;
            }
        }
        while (insertPos < arr.length) {
            arr[insertPos++] = 0;
        }
    }

    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 3, 12};
        moveZeroes(arr);
        for (int x : arr) System.out.print(x + " ");
    }
}
```

EXPLANATION

Overwrite the array from the front with non-zero elements in order, then fill the remaining positions with zero. Maintains relative order of non-zero elements.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q5 Find the Missing Number in an Array (1 to N)

Easy

TRICK / KEY INSIGHT

Use the Gauss sum formula n(n+1)/2 minus actual sum — OR XOR all numbers 1..n with all array elements; the XOR trick avoids overflow for very large n.*

JAVA CODE

```java
public class MissingNumber {
    public static int findMissing(int[] arr, int n) {
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : arr) actualSum += num;
        return expectedSum - actualSum;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 5, 6};   // missing 3, n = 6
        System.out.println("Missing: " + findMissing(arr, 6));
    }
}
```

EXPLANATION

Sum of first n natural numbers is n(n+1)/2. Subtract the actual array sum from this expected sum to get the missing number. XOR method is preferred when sums could overflow.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q6 Find Duplicate Number in Array

Medium

TRICK / KEY INSIGHT

Floyd's Cycle Detection (Tortoise and Hare) works in O(1) space by treating array values as a linked list of indices.

JAVA CODE

```java
public class FindDuplicate {
    public static int findDuplicate(int[] nums) {
        int slow = nums[0];
        int fast = nums[0];

        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);

        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }

    public static void main(String[] args) {
        int[] nums = {1, 3, 4, 2, 2};
        System.out.println("Duplicate: " + findDuplicate(nums));
    }
}
```

EXPLANATION

Since values point to array indices, a duplicate creates a cycle. Floyd's algorithm finds the cycle's entry point, which is the duplicate value. Avoids using extra HashSet space.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q7 Best Time to Buy and Sell Stock (Max Profit)

Easy

TRICK / KEY INSIGHT

Track the minimum price seen so far while scanning left to right; profit = current price - minPrice.

JAVA CODE

```java
public class StockProfit {
    public static int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;

        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }
        return maxProfit;
    }

    public static void main(String[] args) {
        int[] prices = {7, 1, 5, 3, 6, 4};
        System.out.println("Max Profit: " + maxProfit(prices));
    }
}
```

EXPLANATION

One pass: keep the lowest price seen so far, and at every step compute profit if you sold today. Update the max profit accordingly.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q8 Two Sum Problem

Easy

TRICK / KEY INSIGHT

Use a HashMap to store (value -> index) while scanning; check if target - current exists in the map already — avoids the O(n^2) brute force.

JAVA CODE

```java
import java.util.HashMap;

public class TwoSum {
    public static int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int[] result = twoSum(nums, 9);
        System.out.println(result[0] + ", " + result[1]);
    }
}
```

EXPLANATION

Single pass with a HashMap. For each number check if its complement (target - num) was seen before; if yes, we found the pair. Most commonly asked coding interview question overall.

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Q9 Kth Largest Element in an Array

Medium

TRICK / KEY INSIGHT

Use a Min-Heap of size k — the heap's top is always the kth largest after processing all elements.

JAVA CODE

```java
import java.util.PriorityQueue;

public class KthLargest {
    public static int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for (int num : nums) {
            minHeap.add(num);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }
        return minHeap.peek();
    }

    public static void main(String[] args) {
        int[] nums = {3, 2, 1, 5, 6, 4};
        System.out.println("Kth Largest: " + findKthLargest(nums, 2));
    }
}
```

EXPLANATION

Maintain a min-heap of size k. Every time the heap grows beyond k, remove the smallest. After processing all elements, the heap's minimum is the kth largest. QuickSelect gives O(n) average time as an alternative.

⏱ **COMPLEXITY:** Time: O(n log k), Space: O(k)

Q10 Merge Two Sorted Arrays

Easy

TRICK / KEY INSIGHT

Three-pointer merge technique used in Merge Sort — compare front elements of both arrays and pick the smaller one each time.

JAVA CODE

```java
public class MergeSortedArrays {
    public static int[] merge(int[] a, int[] b) {
        int[] result = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;

        while (i < a.length && j < b.length) {
            result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
        }
        while (i < a.length) result[k++] = a[i++];
        while (j < b.length) result[k++] = b[j++];

        return result;
    }

    public static void main(String[] args) {
        int[] a = {1, 3, 5};
        int[] b = {2, 4, 6};
        int[] merged = merge(a, b);
        for (int x : merged) System.out.print(x + " ");
    }
}
```

EXPLANATION

This is the core merge step of Merge Sort. Since both arrays are already sorted, compare the fronts and always take the smaller, then append leftovers.

⏱ **COMPLEXITY:** Time: O(n + m), Space: O(n + m)

Strings

Q11 Check if a String is a Palindrome

Easy

TRICK / KEY INSIGHT

Two-pointer from both ends; also remember StringBuilder(s).reverse().toString().equals(s) as a one-liner trick for interviews.

JAVA CODE

```java
public class PalindromeCheck {
    public static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) return false;
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isPalindrome("madam"));  // true
        System.out.println(isPalindrome("hello"));  // false
    }
}
```

EXPLANATION

Two pointers converge from both ends comparing characters. If any mismatch found, it's not a palindrome. O(1) extra space compared to reversing the string.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q12 Check if Two Strings are Anagrams

Easy

TRICK / KEY INSIGHT

Sort both strings and compare (simple) OR use a frequency array/HashMap of size 26 for O(n).

JAVA CODE

```java
import java.util.Arrays;

public class AnagramCheck {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;

        int[] freq = new int[26];
        for (char c : s1.toCharArray()) freq[c - 'a']++;
        for (char c : s2.toCharArray()) freq[c - 'a']--;

        for (int f : freq) {
            if (f != 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent"));  // true
    }
}
```

EXPLANATION

Count character frequency of the first string, then decrement using the second string's characters. If all counts return to zero, they are anagrams. Faster than sorting (O(n log n)).

⏱ **COMPLEXITY:** Time: O(n), Space: O(1) (fixed 26-size array)

Q13 Reverse Words in a Sentence

Easy

TRICK / KEY INSIGHT

Split by whitespace, then reconstruct while iterating from the last word to the first.

JAVA CODE

```java
public class ReverseWords {
    public static String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i != 0) sb.append(" ");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        System.out.println(reverseWords("the sky is blue"));
        // Output: "blue is sky the"
    }
}
```

EXPLANATION

Split the sentence on whitespace to get individual words, then append them in reverse order using a StringBuilder for efficient concatenation.

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Q14 Find the First Non-Repeating Character

Easy

TRICK / KEY INSIGHT

Use a LinkedHashMap or frequency array first pass to count, second pass to find first count==1.

JAVA CODE

```java
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeating {
    public static char firstNonRepeating(String s) {
        Map<Character, Integer> freq = new LinkedHashMap<>();
        for (char c : s.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }
        for (char c : s.toCharArray()) {
            if (freq.get(c) == 1) return c;
        }
        return '_'; // no unique character
    }

    public static void main(String[] args) {
        System.out.println(firstNonRepeating("swiss"));  // 'w'
    }
}
```

EXPLANATION

First pass builds a frequency map preserving insertion order. Second pass finds the first character whose frequency is exactly 1.

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Q15 Longest Substring Without Repeating Characters

Medium

TRICK / KEY INSIGHT

Sliding window + HashSet/HashMap: expand right pointer, shrink left pointer whenever a duplicate is found inside the window.

JAVA CODE

```java
import java.util.HashMap;

public class LongestUniqueSubstring {
    public static int lengthOfLongestSubstring(String s) {
        HashMap<Character, Integer> map = new HashMap<>();
        int maxLen = 0, left = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (map.containsKey(c) && map.get(c) >= left) {
                left = map.get(c) + 1;
            }
            map.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }

    public static void main(String[] args) {
        System.out.println(lengthOfLongestSubstring("abcabcbb"));  // 3
    }
}
```

EXPLANATION

Sliding window technique. Expand the right edge; when a repeated character is found within the current window, jump the left edge past its last occurrence. Track the max window length throughout.

⏱ **COMPLEXITY:** Time: O(n), Space: O(min(n, charset size))

Q16 Check if String Contains Only Digits / Valid Number

Easy

TRICK / KEY INSIGHT

Use Character.isDigit() in a loop, or the built-in regex s.matches("[0-9]+") for a quick check.

JAVA CODE

```java
public class ValidNumberCheck {
    public static boolean isNumeric(String s) {
        if (s == null || s.isEmpty()) return false;
        for (char c : s.toCharArray()) {
            if (!Character.isDigit(c)) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isNumeric("12345"));  // true
        System.out.println(isNumeric("12a45"));  // false
    }
}
```

EXPLANATION

Iterate through each character and validate using Character.isDigit(). Regex is a quick alternative but is slower for very large strings due to pattern compilation overhead.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Linked List

Q17 Reverse a Linked List

Easy

TRICK / KEY INSIGHT

Three-pointer iterative trick: prev, curr, next — flip curr.next to point backward, then shift all three pointers forward.

JAVA CODE

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public class ReverseLinkedList {
    public static Node reverse(Node head) {
        Node prev = null, curr = head;
        while (curr != null) {
            Node nextTemp = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextTemp;
        }
        return prev;  // new head
    }

    public static void printList(Node head) {
        while (head != null) {
            System.out.print(head.data + " -> ");
            head = head.next;
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        Node head = new Node(1);
        head.next = new Node(2);
        head.next.next = new Node(3);
        head = reverse(head);
        printList(head);  // 3 -> 2 -> 1 -> null
    }
}
```

EXPLANATION

Walk through the list once, redirecting each node's 'next' pointer to the previous node instead of the next. Three pointers (prev, curr, next) are needed to avoid losing the rest of the list. One of THE most asked linked-list questions.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q18 Detect a Cycle in a Linked List

Easy

TRICK / KEY INSIGHT

Floyd's Tortoise and Hare: slow pointer moves 1 step, fast pointer moves 2 steps — if they ever meet, there's a cycle.

JAVA CODE

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public class DetectCycle {
    public static boolean hasCycle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }

    public static void main(String[] args) {
        Node head = new Node(1);
        head.next = new Node(2);
        head.next.next = new Node(3);
        head.next.next.next = head;  // creates a cycle
        System.out.println(hasCycle(head));  // true
    }
}
```

EXPLANATION

Slow pointer moves one node at a time, fast pointer moves two. If a cycle exists, fast eventually laps slow and they meet inside the loop. If fast reaches null, no cycle exists.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q19 Find the Middle of a Linked List

Easy

TRICK / KEY INSIGHT

Slow/fast pointer trick: when fast reaches the end, slow is exactly at the middle.

JAVA CODE

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public class MiddleOfList {
    public static Node findMiddle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    public static void main(String[] args) {
        Node head = new Node(1);
        head.next = new Node(2);
        head.next.next = new Node(3);
        head.next.next.next = new Node(4);
        System.out.println(findMiddle(head).data);  // 3
    }
}
```

EXPLANATION

Fast pointer moves twice as fast as slow. When fast hits the end of the list, slow has covered exactly half the distance, landing on the middle node.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q20 Merge Two Sorted Linked Lists

Easy

TRICK / KEY INSIGHT

Use a dummy head node to simplify edge cases; then simple comparison merge like merging arrays.

JAVA CODE

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public class MergeSortedLists {
    public static Node merge(Node l1, Node l2) {
        Node dummy = new Node(-1);
        Node tail = dummy;

        while (l1 != null && l2 != null) {
            if (l1.data <= l2.data) {
                tail.next = l1;
                l1 = l1.next;
            } else {
                tail.next = l2;
                l2 = l2.next;
            }
            tail = tail.next;
        }
        tail.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

EXPLANATION

A dummy node avoids special-casing the head of the merged list. Walk both lists, always attaching the smaller current node, then attach whatever list remains at the end.

⏱ **COMPLEXITY:** Time: O(n + m), Space: O(1)

Q21 Remove Nth Node From End of List

Medium

TRICK / KEY INSIGHT

One-pass two-pointer trick: move a lead pointer n steps ahead first, then move both pointers together until lead hits the end.

JAVA CODE

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public class RemoveNthFromEnd {
    public static Node removeNth(Node head, int n) {
        Node dummy = new Node(-1);
        dummy.next = head;
        Node fast = dummy, slow = dummy;

        for (int i = 0; i < n; i++) fast = fast.next;

        while (fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }
        slow.next = slow.next.next;  // remove target node
        return dummy.next;
    }
}
```

EXPLANATION

Advance the fast pointer n nodes ahead of slow. Then move both together; when fast reaches the last node, slow sits right before the node to remove. Single pass, no length count needed.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Q22 Detect and Find Start of Cycle

Medium

TRICK / KEY INSIGHT

After slow/fast meet inside the cycle, reset one pointer to head and move both one step at a time — they meet exactly at the cycle's start (Floyd's algorithm, phase 2).

JAVA CODE

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public class CycleStart {
    public static Node detectCycleStart(Node head) {
        Node slow = head, fast = head;
        boolean hasCycle = false;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) { hasCycle = true; break; }
        }
        if (!hasCycle) return null;

        slow = head;
        while (slow != fast) {
            slow = slow.next;
            fast = fast.next;
        }
        return slow;  // start of the cycle
    }
}
```

EXPLANATION

Mathematical property of Floyd's algorithm: the distance from head to cycle start equals the distance from the meeting point to the cycle start (moving forward). This is a very popular 'part 2' follow-up question after basic cycle detection.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Stack & Queue

Q23 Implement Stack Using Array

Easy

TRICK / KEY INSIGHT

Just maintain a 'top' index; push increments then writes, pop reads then decrements.

JAVA CODE

```java
public class ArrayStack {
    private int[] arr;
    private int top;

    public ArrayStack(int size) {
        arr = new int[size];
        top = -1;
    }

    public void push(int x) {
        if (top == arr.length - 1) throw new RuntimeException("Stack Overflow");
        arr[++top] = x;
    }

    public int pop() {
        if (top == -1) throw new RuntimeException("Stack Underflow");
        return arr[top--];
    }

    public int peek() {
        return arr[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }
}
```

EXPLANATION

A simple index-based array implementation. 'top' tracks the current stack top; push writes at top+1, pop reads and decrements. Always guard against overflow/underflow.

⏱ **COMPLEXITY:** Time: O(1) for push/pop/peek, Space: O(n)

Q24 Valid Parentheses / Balanced Brackets

Easy

TRICK / KEY INSIGHT

Push opening brackets onto a stack; on a closing bracket, pop and check it matches — classic stack matching problem.

JAVA CODE

```java
import java.util.Stack;

public class ValidParentheses {
    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if (c == ')' && top != '(') return false;
                if (c == '}' && top != '{') return false;
                if (c == ']' && top != '[') return false;
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(isValid("{[()]}"));  // true
        System.out.println(isValid("{[(])}"));  // false
    }
}
```

EXPLANATION

Push every opening bracket. On a closing bracket, pop the stack top and verify it's the matching opening type. If the stack is empty mid-way or non-empty at the end, it's invalid. Extremely common interview question testing stack fundamentals.

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Q25 Implement Queue Using Two Stacks

Medium

TRICK / KEY INSIGHT

Use an 'in' stack for enqueue and an 'out' stack for dequeue — only transfer elements from in to out when out is empty (amortized O(1)).

JAVA CODE

```java
import java.util.Stack;

public class QueueUsingStacks {
    private Stack<Integer> inStack = new Stack<>();
    private Stack<Integer> outStack = new Stack<>();

    public void enqueue(int x) {
        inStack.push(x);
    }

    public int dequeue() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());
            }
        }
        if (outStack.isEmpty()) throw new RuntimeException("Queue is empty");
        return outStack.pop();
    }
}
```

EXPLANATION

'in' stack handles new elements; when a dequeue is requested and 'out' is empty, dump all of 'in' into 'out', reversing the order so the oldest element is on top. Each element moves at most twice, giving amortized O(1) per operation.

⏱ **COMPLEXITY:** Time: O(1) amortized, Space: O(n)

Q26 Next Greater Element

Medium

TRICK / KEY INSIGHT

Use a monotonic decreasing stack — while the current element is greater than the stack's top, pop and record the current element as its 'next greater'.

JAVA CODE

```java
import java.util.Stack;
import java.util.Arrays;

public class NextGreaterElement {
    public static int[] nextGreater(int[] arr) {
        int n = arr.length;
        int[] result = new int[n];
        Arrays.fill(result, -1);
        Stack<Integer> stack = new Stack<>();  // stores indices

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
                result[stack.pop()] = arr[i];
            }
            stack.push(i);
        }
        return result;
    }

    public static void main(String[] args) {
        int[] arr = {4, 5, 2, 25};
        System.out.println(Arrays.toString(nextGreater(arr)));
        // [5, 25, 25, -1]
    }
}
```

EXPLANATION

Maintain a stack of indices whose 'next greater element' hasn't been found yet. For each new element, pop all smaller elements from the stack since the current element is their answer. Classic monotonic stack pattern used in many variations (stock span, histogram).

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Q27 Min Stack (Get Minimum in O(1)) Medium

TRICK / KEY INSIGHT

Use two stacks: one for actual values, one that tracks the minimum at each level (push min(newVal, currentMin) every time).

JAVA CODE

```java
import java.util.Stack;

public class MinStack {
    private Stack<Integer> stack = new Stack<>();
    private Stack<Integer> minStack = new Stack<>();

    public void push(int x) {
        stack.push(x);
        int currentMin = minStack.isEmpty() ? x : Math.min(x, minStack.peek());
        minStack.push(currentMin);
    }

    public void pop() {
        stack.pop();
        minStack.pop();
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}
```

EXPLANATION

The second stack mirrors the first but always stores the minimum value up to that point. This lets getMin() run in O(1) without scanning, and pop() automatically 'restores' the previous minimum.

⏱ **COMPLEXITY:** Time: O(1) for all operations, Space: O(n)

Trees

Q28 Inorder / Preorder / Postorder Traversal (Recursive)

Easy

TRICK / KEY INSIGHT

Remember the ROOT position in the name: PRE-order = root First, IN-order = root Middle, POST-order = root Last.

JAVA CODE

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class TreeTraversals {
    public static void inorder(TreeNode root) {
        if (root == null) return;
        inorder(root.left);
        System.out.print(root.val + " ");
        inorder(root.right);
    }

    public static void preorder(TreeNode root) {
        if (root == null) return;
        System.out.print(root.val + " ");
        preorder(root.left);
        preorder(root.right);
    }

    public static void postorder(TreeNode root) {
        if (root == null) return;
        postorder(root.left);
        postorder(root.right);
        System.out.print(root.val + " ");
    }
}
```

EXPLANATION

All three are DFS traversals differing only in when the root is visited relative to its children. Inorder on a BST gives sorted order — a very useful fact to mention in interviews.

⏱ **COMPLEXITY:** Time: O(n), Space: O(h) recursion stack, h = tree height

Q29 Level Order Traversal (BFS)

Easy

TRICK / KEY INSIGHT

Use a Queue: process one level fully before moving to the next by tracking the queue's size at the start of each level.

JAVA CODE

```java
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class LevelOrderTraversal {
    public static List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.add(node.left);
                if (node.right != null) queue.add(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

EXPLANATION

Standard BFS using a queue. Capturing queue.size() before the inner loop is the trick that lets you group nodes level by level instead of just flattening the whole tree.

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Q30 Find the Height / Maximum Depth of a Binary Tree

Easy

TRICK / KEY INSIGHT

height(node) = 1 + max(height(left), height(right)) — classic bottom-up recursion.

JAVA CODE

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class TreeHeight {
    public static int height(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(height(root.left), height(root.right));
    }
}
```

EXPLANATION

Base case: an empty tree has height 0. Recursive case: the height is 1 (for the current node) plus the taller of its two subtrees.

⏱ **COMPLEXITY:** Time: O(n), Space: O(h)

Q31 Check if a Binary Tree is Balanced

Medium

TRICK / KEY INSIGHT

Combine height calculation with balance checking in ONE pass — return -1 as a sentinel the moment imbalance is detected to short-circuit further recursion.

JAVA CODE

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class BalancedTreeCheck {
    public static boolean isBalanced(TreeNode root) {
        return checkHeight(root) != -1;
    }

    private static int checkHeight(TreeNode node) {
        if (node == null) return 0;

        int leftHeight = checkHeight(node.left);
        if (leftHeight == -1) return -1;

        int rightHeight = checkHeight(node.right);
        if (rightHeight == -1) return -1;

        if (Math.abs(leftHeight - rightHeight) > 1) return -1;

        return 1 + Math.max(leftHeight, rightHeight);
    }
}
```

EXPLANATION

Naive approach recomputes height repeatedly (O(n^2)). The trick is to fold the balance check into the same recursive call that computes height, using -1 as an early-exit signal, bringing it down to O(n).

⏱ **COMPLEXITY:** Time: O(n), Space: O(h)

Q32 Lowest Common Ancestor (LCA) in a Binary Tree

Medium

TRICK / KEY INSIGHT

If the current node is null or matches either target, return it. Otherwise recurse both sides — if both sides return non-null, the current node IS the LCA.

JAVA CODE

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class LowestCommonAncestor {
    public static TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;

        TreeNode left = lca(root.left, p, q);
        TreeNode right = lca(root.right, p, q);

        if (left != null && right != null) return root;  // split point found
        return (left != null) ? left : right;
    }
}
```

EXPLANATION

Post-order style recursion. If p and q are found on different sides of a node, that node is their lowest common ancestor. If both are on the same side, the LCA lies deeper in that subtree.

⏱ **COMPLEXITY:** Time: O(n), Space: O(h)

Q33 Validate Binary Search Tree (BST)

Medium

TRICK / KEY INSIGHT

Don't just compare each node to its immediate children — pass down a valid (min, max) RANGE that shrinks as you recurse deeper.

JAVA CODE

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class ValidateBST {
    public static boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private static boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;

        return validate(node.left, min, node.val) &&
               validate(node.right, node.val, max);
    }
}
```

EXPLANATION

A common mistake is only checking a node against its direct parent. The correct approach passes an allowed (min, max) range downward — every node in the left subtree must be less than ALL its ancestors up to the point a right turn was made, not just its parent.

⏱ **COMPLEXITY:** Time: O(n), Space: O(h)

Q34 Diameter of a Binary Tree

Medium

TRICK / KEY INSIGHT

The diameter at any node = leftHeight + rightHeight (edges through that node); track a global max while computing height recursively.

JAVA CODE

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class TreeDiameter {
    static int maxDiameter = 0;

    public static int diameter(TreeNode root) {
        maxDiameter = 0;
        height(root);
        return maxDiameter;
    }

    private static int height(TreeNode node) {
        if (node == null) return 0;
        int left = height(node.left);
        int right = height(node.right);
        maxDiameter = Math.max(maxDiameter, left + right);
        return 1 + Math.max(left, right);
    }
}
```

EXPLANATION

The diameter (longest path between any two nodes) doesn't have to pass through the root. By computing height bottom-up and updating a global max at every node with left+right height, you capture the true diameter in a single traversal.

⏱ **COMPLEXITY:** Time: O(n), Space: O(h)

Q35 Serialize and Deserialize a Binary Tree

Hard

TRICK / KEY INSIGHT

Use preorder traversal with explicit 'null' markers for missing children so the tree structure can be perfectly reconstructed.

JAVA CODE

```java
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class SerializeTree {
    public static String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        buildString(root, sb);
        return sb.toString();
    }

    private static void buildString(TreeNode node, StringBuilder sb) {
        if (node == null) {
            sb.append("null,");
            return;
        }
        sb.append(node.val).append(",");
        buildString(node.left, sb);
        buildString(node.right, sb);
    }

    public static TreeNode deserialize(String data) {
        Queue<String> nodes = new LinkedList<>(Arrays.asList(data.split(",")));
        return buildTree(nodes);
    }

    private static TreeNode buildTree(Queue<String> nodes) {
        String val = nodes.poll();
        if (val.equals("null")) return null;
        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = buildTree(nodes);
        node.right = buildTree(nodes);
        return node;
    }
}
```

EXPLANATION

Preorder (root-left-right) with explicit null markers preserves full structural information, so deserialization can rebuild the exact tree by consuming tokens in the same order they were written. A favorite hard-level question at top companies.

⏱ **COMPLEXITY:** Time: O(n), Space: O(n)

Graphs

Q36 Graph Representation (Adjacency List)

Easy

TRICK / KEY INSIGHT

Use ArrayList<ArrayList<Integer>> or a HashMap<Integer, List<Integer>> — adjacency list is preferred over matrix for sparse graphs (saves space).

JAVA CODE

```java
import java.util.*;

public class GraphAdjList {
    public static List<List<Integer>> buildGraph(int n, int[][] edges) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());

        for (int[] edge : edges) {
            adj.get(edge[0]).add(edge[1]);
            adj.get(edge[1]).add(edge[0]);  // remove this line for directed graph
        }
        return adj;
    }
}
```

EXPLANATION

An adjacency list stores, for each vertex, a list of its neighbors. It uses O(V + E) space compared to O(V^2) for an adjacency matrix, making it ideal for sparse graphs (most real-world and interview graphs).

⏱ **COMPLEXITY:** Space: O(V + E)

Q37 BFS Traversal of a Graph

Easy

TRICK / KEY INSIGHT

Use a Queue + a visited[] boolean array — mark visited AT THE TIME OF ENQUEUEING, not dequeueing, to avoid duplicate enqueues.

JAVA CODE

```java
import java.util.*;

public class GraphBFS {
    public static List<Integer> bfs(int start, List<List<Integer>> adj, int n) {
        List<Integer> result = new ArrayList<>();
        boolean[] visited = new boolean[n];
        Queue<Integer> queue = new LinkedList<>();

        queue.add(start);
        visited[start] = true;

        while (!queue.isEmpty()) {
            int node = queue.poll();
            result.add(node);

            for (int neighbor : adj.get(node)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.add(neighbor);
                }
            }
        }
        return result;
    }
}
```

EXPLANATION

BFS explores level by level using a queue. Marking a node visited as soon as it's enqueued (not when dequeued) prevents the same node from being added multiple times before it's processed.

⏱ **COMPLEXITY:** Time: O(V + E), Space: O(V)

Q38 DFS Traversal of a Graph

Easy

TRICK / KEY INSIGHT

Recursive DFS is simplest: mark current node visited, then recurse into every unvisited neighbor. For iterative DFS, use an explicit Stack.

JAVA CODE

```java
import java.util.*;

public class GraphDFS {
    public static void dfs(int node, List<List<Integer>> adj, boolean[] visited, List<Integer>
result) {
        visited[node] = true;
        result.add(node);

        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, adj, visited, result);
            }
        }
    }

    public static List<Integer> dfsTraversal(int start, List<List<Integer>> adj, int n) {
        boolean[] visited = new boolean[n];
        List<Integer> result = new ArrayList<>();
        dfs(start, adj, visited, result);
        return result;
    }
}
```

EXPLANATION

DFS dives as deep as possible along each branch before backtracking. The recursive call stack naturally handles the backtracking for you. Foundation for cycle detection, topological sort, and connected components.

⏱ **COMPLEXITY:** Time: O(V + E), Space: O(V)

Q39 Detect Cycle in a Directed Graph

Medium

TRICK / KEY INSIGHT

Track TWO states: visited (ever seen) and inRecursionStack (currently on the DFS path). A cycle exists if you revisit a node that's still in the recursion stack.

JAVA CODE

```java
import java.util.*;

public class DetectCycleDirected {
    public static boolean hasCycle(int n, List<List<Integer>> adj) {
        boolean[] visited = new boolean[n];
        boolean[] inStack = new boolean[n];

        for (int i = 0; i < n; i++) {
            if (!visited[i] && dfs(i, adj, visited, inStack)) {
                return true;
            }
        }
        return false;
    }

    private static boolean dfs(int node, List<List<Integer>> adj, boolean[] visited, boolean[]
inStack) {
        visited[node] = true;
        inStack[node] = true;

        for (int neighbor : adj.get(node)) {
            if (inStack[neighbor]) return true;   // back edge = cycle
            if (!visited[neighbor] && dfs(neighbor, adj, visited, inStack)) return true;
        }

        inStack[node] = false;  // backtrack
        return false;
    }
}
```

EXPLANATION

A simple 'visited' array alone is not enough for directed graphs — you also need to know if a node is on the CURRENT DFS path (inStack). Finding an edge back to a node still on the path means there's a cycle. Remove from inStack when backtracking.

⏱ **COMPLEXITY:** Time: O(V + E), Space: O(V)

Q40 Topological Sort (Kahn's Algorithm — BFS based) Medium

TRICK / KEY INSIGHT

Compute in-degree of every node; repeatedly remove nodes with in-degree 0, decrementing their neighbors' in-degree — this naturally gives a valid dependency order.

JAVA CODE

```java
import java.util.*;

public class TopologicalSort {
    public static List<Integer> topoSort(int n, List<List<Integer>> adj) {
        int[] inDegree = new int[n];
        for (List<Integer> neighbors : adj) {
            for (int v : neighbors) inDegree[v]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) queue.add(i);
        }

        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
            int node = queue.poll();
            result.add(node);
            for (int neighbor : adj.get(node)) {
                if (--inDegree[neighbor] == 0) {
                    queue.add(neighbor);
                }
            }
        }
        return result;  // if result.size() < n, a cycle exists
    }
}
```

EXPLANATION

Nodes with in-degree 0 have no unmet dependencies, so they can go first. Removing them (conceptually) reduces the in-degree of their neighbors, exposing new zero in-degree nodes. Used for build systems, course scheduling, task ordering.

⏱ **COMPLEXITY:** Time: O(V + E), Space: O(V)

Q41 Number of Islands (Connected Components in a Grid) Medium

TRICK / KEY INSIGHT

Flood-fill with DFS or BFS from every unvisited land cell; each flood-fill call finds one full island, so just count how many times you start a new flood-fill.

JAVA CODE

```java
public class NumberOfIslands {
    public static int numIslands(char[][] grid) {
        int count = 0;
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    dfs(grid, i, j);
                }
            }
        }
        return count;
    }

    private static void dfs(char[][] grid, int i, int j) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] != '1') {
            return;
        }
        grid[i][j] = '0';  // mark as visited
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
}
```

EXPLANATION

Treat the grid as an implicit graph where each land cell connects to its 4 neighbors. Every time a new unvisited '1' is found, it's a brand-new island — flood-fill (DFS/BFS) sinks the entire island so it's not counted again.

⏱ **COMPLEXITY:** Time: O(rows * cols), Space: O(rows * cols) worst case recursion

Q42 Dijkstra's Algorithm (Shortest Path)

Medium

TRICK / KEY INSIGHT

Use a PriorityQueue (min-heap) on distance — always expand the closest unvisited node next; relax edges only if a shorter path is found.

JAVA CODE

```java
import java.util.*;

public class DijkstraAlgorithm {
    public static int[] dijkstra(int n, List<List<int[]>> adj, int src) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.add(new int[]{src, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int node = curr[0], d = curr[1];
            if (d > dist[node]) continue;  // stale entry

            for (int[] edge : adj.get(node)) {
                int neighbor = edge[0], weight = edge[1];
                if (dist[node] + weight < dist[neighbor]) {
                    dist[neighbor] = dist[node] + weight;
                    pq.add(new int[]{neighbor, dist[neighbor]});
                }
            }
        }
        return dist;
    }
}
```

EXPLANATION

Greedy shortest path algorithm for graphs with non-negative weights. The min-heap always gives you the currently-closest unvisited node, so once popped its distance is final. Skipping stale (outdated) queue entries keeps it efficient.

⏱ **COMPLEXITY:** Time: O((V + E) log V), Space: O(V)

Sorting

Q43 Bubble Sort

Easy

TRICK / KEY INSIGHT

Add an 'isSwapped' flag — if no swaps happen in a full pass, the array is already sorted, so you can break early (best case becomes O(n)).

JAVA CODE

```java
public class BubbleSort {
    public static void sort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
    }
}
```

EXPLANATION

Repeatedly swap adjacent out-of-order elements. Each full pass 'bubbles' the largest remaining element to its correct position at the end.

⏱ **COMPLEXITY:** Time: O(n^2) worst/avg, O(n) best with early exit; Space: O(1)

Q44 Quick Sort

Medium

TRICK / KEY INSIGHT

Pick a pivot (commonly last element), partition array so smaller elements go left and larger go right, then recursively sort each partition — remember the Lomuto partition scheme.

JAVA CODE

```java
public class QuickSort {
    public static void sort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            sort(arr, low, pi - 1);
            sort(arr, pi + 1, high);
        }
    }

    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
            }
        }
        int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
        return i + 1;
    }
}
```

EXPLANATION

Divide and conquer: partition the array around a pivot so everything smaller is on the left and everything larger on the right, then recursively apply to each side. In-place and very fast in practice despite O(n^2) worst case.

⏱ **COMPLEXITY:** Time: O(n log n) avg, O(n^2) worst; Space: O(log n)

Q45 Merge Sort

Medium

TRICK / KEY INSIGHT

Split the array in half recursively until size 1, then merge sorted halves back together — guaranteed O(n log n) regardless of input, unlike Quick Sort.

JAVA CODE

```java
public class MergeSort {
    public static void sort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            sort(arr, left, mid);
            sort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    private static void merge(int[] arr, int left, int mid, int right) {
        int[] temp = new int[right - left + 1];
        int i = left, j = mid + 1, k = 0;

        while (i <= mid && j <= right) {
            temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
        }
        while (i <= mid) temp[k++] = arr[i++];
        while (j <= right) temp[k++] = arr[j++];

        System.arraycopy(temp, 0, arr, left, temp.length);
    }
}
```

EXPLANATION

Classic divide-and-conquer: recursively split until single elements, then merge sorted halves back together. Stable and guarantees O(n log n) even in the worst case, but uses extra O(n) space, unlike in-place Quick Sort.

⏱ **COMPLEXITY:** Time: O(n log n) always; Space: O(n)

Q46 Binary Search

Easy

TRICK / KEY INSIGHT

Always compute mid as low + (high - low) / 2 (NOT (low+high)/2) to avoid integer overflow on large arrays.

JAVA CODE

```java
public class BinarySearch {
    public static int search(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) return mid;
            else if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
}
```

EXPLANATION

Repeatedly halve the search space by comparing the middle element to the target. Works only on sorted arrays. The overflow-safe mid calculation is a small but important detail interviewers look for.

⏱ **COMPLEXITY:** Time: O(log n), Space: O(1)

Q47 Search in a Rotated Sorted Array

Medium

TRICK / KEY INSIGHT

Modified binary search: at each step, figure out which HALF is properly sorted, then check if target lies within that sorted half's range.

JAVA CODE

```java
public class SearchRotatedArray {
    public static int search(int[] arr, int target) {
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) return mid;

            if (arr[low] <= arr[mid]) {  // left half sorted
                if (arr[low] <= target && target < arr[mid]) high = mid - 1;
                else low = mid + 1;
            } else {  // right half sorted
                if (arr[mid] < target && target <= arr[high]) low = mid + 1;
                else high = mid - 1;
            }
        }
        return -1;
    }
}
```

EXPLANATION

A rotated sorted array always has at least one half (left or right of mid) that is properly sorted. Determine which half is sorted, then check if the target falls in that half's range to decide which direction to search next.

⏱ **COMPLEXITY:** Time: O(log n), Space: O(1)

Q48 Find First and Last Position of Element (Binary Search Variant) Medium

TRICK / KEY INSIGHT

Run binary search TWICE with a bias: once biased to find the leftmost occurrence, once biased to find the rightmost — don't stop at the first match found.

JAVA CODE

```java
public class FirstLastPosition {
    public static int[] searchRange(int[] arr, int target) {
        return new int[]{findBound(arr, target, true), findBound(arr, target, false)};
    }

    private static int findBound(int[] arr, int target, boolean findFirst) {
        int low = 0, high = arr.length - 1, result = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) {
                result = mid;
                if (findFirst) high = mid - 1;   // keep searching left
                else low = mid + 1;               // keep searching right
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }
}
```

EXPLANATION

Standard binary search stops as soon as it finds a match. Here, even after finding a match, we keep narrowing in one direction (left for first occurrence, right for last) to squeeze out the boundary.

⏱ **COMPLEXITY:** Time: O(log n), Space: O(1)

Q49 Sort an Array of 0s, 1s, and 2s (Dutch National Flag)

Medium

TRICK / KEY INSIGHT

Three pointers: low, mid, high. Swap 0s to the front, 2s to the back, and leave 1s in the middle — single pass, no counting needed.

JAVA CODE

```java
public class SortColors {
    public static void sortColors(int[] arr) {
        int low = 0, mid = 0, high = arr.length - 1;

        while (mid <= high) {
            if (arr[mid] == 0) {
                swap(arr, low++, mid++);
            } else if (arr[mid] == 1) {
                mid++;
            } else {
                swap(arr, mid, high--);
            }
        }
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
}
```

EXPLANATION

Dutch National Flag algorithm. Three regions are maintained: [0..low) are 0s, [low..mid) are 1s, (high..end] are 2s. Single pass with constant space — much better than a two-pass counting sort.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Dynamic Programming

Q50 Fibonacci Number (Memoization & Tabulation)

Easy

TRICK / KEY INSIGHT

Always mention BOTH approaches in interviews: top-down memoization (recursion + cache) and bottom-up tabulation (iterative array) — tabulation avoids recursion stack overhead.

JAVA CODE

```java
import java.util.HashMap;

public class Fibonacci {
    // Top-down (Memoization)
    public static int fibMemo(int n, HashMap<Integer, Integer> memo) {
        if (n <= 1) return n;
        if (memo.containsKey(n)) return memo.get(n);
        int result = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
        memo.put(n, result);
        return result;
    }

    // Bottom-up (Tabulation)
    public static int fibTab(int n) {
        if (n <= 1) return n;
        int[] dp = new int[n + 1];
        dp[0] = 0; dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }

    // Space-optimized O(1)
    public static int fibOptimized(int n) {
        if (n <= 1) return n;
        int prev2 = 0, prev1 = 1;
        for (int i = 2; i <= n; i++) {
            int curr = prev1 + prev2;
            prev2 = prev1;
            prev1 = curr;
        }
        return prev1;
    }
}
```

EXPLANATION

The naive recursive Fibonacci recomputes the same subproblems exponentially many times (O(2^n)). Memoization caches results; tabulation builds them bottom-up. Since Fibonacci only needs the last two values, space can be optimized to O(1).

⏱ **COMPLEXITY:** Time: O(n), Space: O(n) memo/tab, O(1) optimized

Q51 0/1 Knapsack Problem

Medium

TRICK / KEY INSIGHT

dp[i][w] = max value using first i items with capacity w. At each item, decide: skip it, or take it (if it fits) — take the better of the two choices.

JAVA CODE

```java
public class Knapsack01 {
    public static int knapsack(int[] weights, int[] values, int capacity) {
        int n = weights.length;
        int[][] dp = new int[n + 1][capacity + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 0; w <= capacity; w++) {
                if (weights[i - 1] <= w) {
                    dp[i][w] = Math.max(
                        values[i - 1] + dp[i - 1][w - weights[i - 1]],  // take
                        dp[i - 1][w]                                     // skip
                    );
                } else {
                    dp[i][w] = dp[i - 1][w];  // can't fit, must skip
                }
            }
        }
        return dp[n][capacity];
    }
}
```

EXPLANATION

Classic 2D DP. Each cell dp[i][w] represents the best value achievable using the first i items within capacity w. The recurrence compares taking vs skipping the current item. Foundation for many DP problems (subset sum, partition, coin change variants).

⏱ **COMPLEXITY:** Time: O(n * capacity), Space: O(n * capacity), optimizable to O(capacity)

Q52 Longest Common Subsequence (LCS) Medium

TRICK / KEY INSIGHT

dp[i][j] = LCS length of s1[0..i) and s2[0..j). If characters match, extend diagonal + 1; else take the max of skipping a character from either string.

JAVA CODE

```java
public class LongestCommonSubsequence {
    public static int lcs(String s1, String s2) {
        int m = s1.length(), n = s2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

EXPLANATION

2D DP table where dp[i][j] holds LCS length for prefixes of length i and j. Matching characters extend the diagonal value; mismatches inherit the best from either shrinking the first or second string. Base for diff tools, DNA sequence alignment.

⏱ **COMPLEXITY:** Time: O(m * n), Space: O(m * n)

Q53 Coin Change (Minimum Coins)

Medium

TRICK / KEY INSIGHT

dp[amount] = minimum coins to make that amount; initialize with 'infinity' (amount+1) and update dp[i] = min(dp[i], dp[i - coin] + 1) for every coin that fits.

JAVA CODE

```java
import java.util.Arrays;

public class CoinChange {
    public static int minCoins(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);  // "infinity"
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

EXPLANATION

Bottom-up DP where dp[i] is the fewest coins needed for amount i. For every amount, try every coin and take the best result of 'use this coin + best solution for the remainder'.

⏱ **COMPLEXITY:** Time: O(amount * coins.length), Space: O(amount)

Q54 Longest Increasing Subsequence (LIS) — Medium

TRICK / KEY INSIGHT

dp[i] = length of LIS ending at index i. For each i, check all previous j < i where arr[j] < arr[i] and take dp[i] = max(dp[i], dp[j]+1). Binary search gives O(n log n).

JAVA CODE

```java
import java.util.Arrays;

public class LongestIncreasingSubsequence {
    public static int lengthOfLIS(int[] arr) {
        int n = arr.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int maxLen = 1;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (arr[j] < arr[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            maxLen = Math.max(maxLen, dp[i]);
        }
        return maxLen;
    }
}
```

EXPLANATION

O(n^2) DP: dp[i] tracks the longest increasing subsequence ending exactly at index i. Mention the O(n log n) optimization using patience sorting + binary search as a strong follow-up in interviews.

⏱ **COMPLEXITY:** Time: O(n^2) (O(n log n) optimized), Space: O(n)

Q55 Climbing Stairs (Distinct Ways) — Easy

TRICK / KEY INSIGHT

This IS Fibonacci in disguise: ways(n) = ways(n-1) + ways(n-2), because you can arrive at step n from either step n-1 (1 step) or step n-2 (2 steps).

JAVA CODE

```java
public class ClimbingStairs {
    public static int climbStairs(int n) {
        if (n <= 2) return n;
        int prev2 = 1, prev1 = 2;
        for (int i = 3; i <= n; i++) {
            int curr = prev1 + prev2;
            prev2 = prev1;
            prev1 = curr;
        }
        return prev1;
    }
}
```

EXPLANATION

Recognizing that this maps directly to the Fibonacci recurrence is the key insight. Space-optimized to O(1) by only tracking the last two values instead of a full DP array.

⏱ **COMPLEXITY:** Time: O(n), Space: O(1)

Recursion & Backtracking

Q56 Generate All Subsets (Power Set)

Medium

TRICK / KEY INSIGHT

At each element, make a binary choice: include it or exclude it — this naturally forms a recursion tree of depth n with 2^n leaves.

JAVA CODE

```java
import java.util.*;

public class Subsets {
    public static List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }

    private static void backtrack(int[] nums, int index, List<Integer> curr, List<List<Integer>>
result) {
        if (index == nums.length) {
            result.add(new ArrayList<>(curr));
            return;
        }
        // exclude nums[index]
        backtrack(nums, index + 1, curr, result);
        // include nums[index]
        curr.add(nums[index]);
        backtrack(nums, index + 1, curr, result);
        curr.remove(curr.size() - 1);  // backtrack
    }
}
```

EXPLANATION

Every element has two choices: in or out of the current subset. Branching on both choices for every element yields all 2^n subsets. Always remember to 'undo' (backtrack) after the include branch.

⏱ **COMPLEXITY:** Time: O(2^n), Space: O(n) recursion depth

Q57 Permutations of an Array

Medium

TRICK / KEY INSIGHT

Swap-based backtracking: fix one element at the current position, recurse on the rest, then SWAP BACK to restore state before trying the next fix.

JAVA CODE

```java
import java.util.*;

public class Permutations {
    public static List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, result);
        return result;
    }

    private static void backtrack(int[] nums, int start, List<List<Integer>> result) {
        if (start == nums.length) {
            List<Integer> perm = new ArrayList<>();
            for (int n : nums) perm.add(n);
            result.add(perm);
            return;
        }
        for (int i = start; i < nums.length; i++) {
            swap(nums, start, i);
            backtrack(nums, start + 1, result);
            swap(nums, start, i);  // backtrack: undo the swap
        }
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
}
```

EXPLANATION

For each position, try placing every remaining unused element there (via swap), recurse to fill the rest, then swap back to restore the original order for the next iteration of the loop. This avoids needing an extra 'used' array.

⏱ **COMPLEXITY:** Time: O(n * n!), Space: O(n)

Q58 N-Queens Problem

Hard

TRICK / KEY INSIGHT

Track attacked columns and both diagonals using boolean sets (col, diag1 = row-col, diag2 = row+col) for O(1) safety checks instead of scanning the board every time.

JAVA CODE

```java
import java.util.*;

public class NQueens {
    public static int solveNQueens(int n) {
        Set<Integer> cols = new HashSet<>();
        Set<Integer> diag1 = new HashSet<>();  // row - col
        Set<Integer> diag2 = new HashSet<>();  // row + col
        return backtrack(0, n, cols, diag1, diag2);
    }

    private static int backtrack(int row, int n, Set<Integer> cols, Set<Integer> diag1,
Set<Integer> diag2) {
        if (row == n) return 1;  // found a valid arrangement

        int count = 0;
        for (int col = 0; col < n; col++) {
            int d1 = row - col, d2 = row + col;
            if (cols.contains(col) || diag1.contains(d1) || diag2.contains(d2)) continue;

            cols.add(col); diag1.add(d1); diag2.add(d2);
            count += backtrack(row + 1, n, cols, diag1, diag2);
            cols.remove(col); diag1.remove(d1); diag2.remove(d2);  // backtrack
        }
        return count;
    }
}
```

EXPLANATION

Place queens row by row. Instead of checking the whole board for attacks (O(n) per check), track which columns and diagonals are occupied in O(1) hash sets. Classic backtracking with constraint propagation — a top hard-level interview favorite.

⏱ **COMPLEXITY:** Time: O(n!), Space: O(n)

Q59 Sudoku Solver

Hard

TRICK / KEY INSIGHT

For each empty cell, try digits 1-9; if a digit is valid (no conflict in row/col/box), place it and recurse — if the recursion fails, backtrack and try the next digit.

JAVA CODE

```java
public class SudokuSolver {
    public static boolean solve(char[][] board) {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == '.') {
                    for (char c = '1'; c <= '9'; c++) {
                        if (isValid(board, row, col, c)) {
                            board[row][col] = c;
                            if (solve(board)) return true;
                            board[row][col] = '.';  // backtrack
                        }
                    }
                    return false;  // no valid digit works here
                }
            }
        }
        return true;  // board fully filled
    }

    private static boolean isValid(char[][] board, int row, int col, char c) {
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == c) return false;
            if (board[i][col] == c) return false;
            int boxRow = 3 * (row / 3) + i / 3;
            int boxCol = 3 * (col / 3) + i % 3;
            if (board[boxRow][boxCol] == c) return false;
        }
        return true;
    }
}
```

EXPLANATION

Classic constraint-satisfaction backtracking. Try each digit in an empty cell; if placing it doesn't violate row/column/box rules, recurse deeper. If a full solution can't be reached, undo (backtrack) and try the next digit.

⏱ **COMPLEXITY:** Time: O(9^(empty cells)) worst case, Space: O(1) extra (in-place)